

Reinforcement Learning

Revision Notes

A complete pass through the theory — MDPs, Monte Carlo, temporal difference, DQN, policy gradients, the neuroscience links, and where the field is still stuck — with every definition, formula, and corner case flagged for review.

Compiled for Akshat · B.Tech, AI & Data Science

Source video: <https://www.youtube.com/watch?v=VnpRp7ZglfA>

Contents

01 Overview & Motivation	3
02 Agent, Environment & the Boundary	4
03 The MDP Formalism	4
04 The Markov Property	5
05 Limits of the MDP Framing	5
06 Policy π and Return G	5
07 Worked Example: Grid Maze	7
08 World Models	7
09 Sampling & Trajectories	7
10 The Monte Carlo Method	8
11 Why Raw Returns Aren't Enough	8
12 Value Functions: V and Q	8
13 Generalized Policy Iteration & ϵ-greedy	9
14 Temporal Difference (TD)	11
15 SARSA, Expected SARSA, Q-learning	11
16 The Bellman Optimality Equation	12
17 Deep Q-Networks (DQN)	14
18 Policy Gradients	16
19 The Objective Function $J(\theta)$	16
20 "How Good Was This Action?" — Advantage & Variance	17
21 Discrete vs. Continuous Policy-Gradient Mechanics	17
22 Actor-Critic, TRPO & PPO	18
23 RL & Neuroscience	19
24 Dopamine & the TD Error	19
25 Actor-Critic in the Brain	20
26 Limitations of Current RL	21
27 Future Direction 1 — Model-Based RL	21
28 Future Direction 2 — Imitation & Inverse RL	23
29 Recap	25
30 Glossary — Quick Reference	26
31 Formula Sheet	29
32 Corner-Case Checklist — Pre-Exam Scan	30

How to use these notes

Teal boxes are formal definitions worth knowing cold. **Rust boxes** flag corner cases and common mix-ups — the things exam and interview questions like to target. A green ► link under a major heading jumps to that exact moment in the source video.

01 Overview & Motivation

► Watch this part — 0:05

Computers do exactly what they're told — which is a blessing for something like adding two huge numbers, and a curse for everyday physical skills. Tasks like tying a shoe, folding a t-shirt, or walking can't be hand-specified: there's no way to write down the exact joint torques, or the exact forces, masses and friction of the physical world, in advance.

DEFINITION

Reinforcement Learning (RL) is the subfield of machine learning where an **agent** learns by trial and error, using positive/negative **reward** to reinforce good behaviour and discourage bad behaviour — closely resembling how humans and animals learn in the physical world.

Milestones to remember

- **2013** — Atari games learned directly from pixels (Deep Q-Networks).
- **2016** — AlphaGo defeats top human Go players.
- **2018** — OpenAI Five defeats professional Dota 2 players.
- **2019** — OpenAI solves a Rubik's Cube with a robot hand.
- Later: robot dogs, self-driving components, and fine-tuning language models (RLHF) all build on RL.

Core sources referenced

- *Reinforcement Learning: An Introduction* — Sutton & Barto (the field's standard text).
- OpenAI's "Spinning Up" resource.

CORNER CASE — notation

This source indexes the reward at the **same** time step as the state/action that produced it: $s_0 a_0 r_0 s_1 a_1 r_1 \dots$. Many other texts (including Sutton & Barto) instead write $s_0 a_0 r_1 s_1 a_1 r_2 \dots$ — i.e. the reward is "one step ahead." When comparing formulas across sources, check which convention is used, or subscripts will look off by one. Some sources also use uppercase R (reward as a random variable) instead of lowercase r .

02 Agent, Environment & the Boundary

► Watch this part — 4:29

DEFINITION

Agent = whatever you can *directly* control. **Environment** = whatever you cannot directly control but are trying to influence indirectly through the agent.

Agent	Environment
Car	Road
Game character	Map
Robot hand	Cube
Chess strategy	The match

- **Action** — the agent's influence on the environment (its “motor output”).
- **State** — the environment's influence on the agent (its “sensory input”).

CORNER CASE — the boundary is arbitrary

Where you draw the agent/environment line depends entirely on your realm of direct control and the task at hand — and it can be nested. Example: learning to drive from zero. **Program A** — agent = brain, environment = your own limbs; actions = neuron signals; state = joint angles via proprioception. **Program B** — agent = you-as-a-creature, environment = the car; actions = limb movements; state = speedometer, steering feel, pedals. **Program C** — agent = the whole car, environment = the road; actions = how the car moves; state = other cars, lanes, speed limit. The time-scale of an “action” is equally arbitrary (closing a fist = one action, or five, one per finger?). This flexibility is a feature: frame the problem however gives it the most useful structure.

03 The MDP Formalism

DEFINITION

Reward — a single scalar signal after each action; more positive = better, more negative = worse. By convention it flows **environment** → **agent**.

DEFINITION

Markov Decision Process (MDP) — a discrete, repeating sequence *state, action, reward, state, action, reward, ...* It either continues forever (a **continuing task**) or terminates (an **episode**, e.g. dying in a game level).

04 The Markov Property

DEFINITION

Markov property — each state depends only on the *immediately preceding* state (and action), not on any earlier history. The current state must be a sufficient summary of everything relevant that came before it.

CORNER CASE — worked example

A robot catching a ball. If the state is **position only**, the next state is *not* determined by the current one — you can't tell direction or speed, so the property is violated. Add **velocity** to the state and the property holds again — even though the robot still doesn't know the underlying physics (gravity, etc.), there is now, in principle, enough information to learn from. Whether the Markov property holds is a design choice about *what you put in the state*, not a fixed fact about the world.

05 Limits of the MDP Framing

- Most RL methods lean on the Markov property to justify using only the most recent state — violate it, and those tools work worse.
- A single fixed time-scale is awkward when a task genuinely spans multiple scales (individual muscle twitches vs. “which city to drive to”).
- A single scalar reward can be limiting for tasks better evaluated along multiple axes at once.

Despite this, essentially all of modern RL is still built on the MDP foundation — no widely-adopted replacement exists yet.

06 Policy π and Return G

DEFINITION

Policy, $\pi(a|s)$ — the probability of taking action a given state s . Written as a conditional probability (not a plain function) specifically so it can be **stochastic** — randomness is what allows exploration of different strategies.

DEFINITION

Return, G_t — the total *discounted* cumulative reward from time t onward.

RETURN

$$G(t) = r(t) + \gamma \cdot r(t+1) + \gamma^2 \cdot r(t+2) + \dots = \sum_{(k \geq 0)} \gamma^k \cdot r(t+k)$$

Discount factor $\gamma \in [0, 1]$ — lowers the importance of rewards further in the future (useful, since the future is harder to predict).

CORNER CASE

$\gamma = 1$ gives an *undiscounted* sum — this only makes sense for **terminating** episodes. For a continuing (never-ending) task, $\gamma = 1$ makes the sum diverge to infinity.

GOAL OF RL

Find the policy π that maximises the expected return.

07 Worked Example: Grid Maze

► Watch this part — 16:38

A simple maze (one wall) demonstrates every key idea because each state is just one grid cell — easy to visualise. The agent moves up/down/left/right, can't pass through walls; an episode ends when it reaches the target **or** after 20 steps, whichever comes first.

- **State** — the (x, y) coordinate, two numbers.
- **Action** — a single integer in $\{0, 1, 2, 3\}$ (up/down/left/right).
- **Reward** — 0 on entering the target cell, -1 otherwise — shaping the goal into “reach the target as fast as possible.”

08 World Models

DEFINITION

World model — a function P giving the (probability of the) next state and reward given the current state and action: $P(s', r | s, a)$. Having access to this is equivalent to “understanding how the environment works.”

DEFINITION

Model-based RL — the agent has or learns a world model. **Model-free** RL — the agent has no such access; states, actions and rewards are just meaningless numbers it must learn to associate with outcomes purely from experience.

Whether an agent has a world model makes a massive difference to learning efficiency — exactly why model-based RL is a major, active research direction (see §27). The grid, DQN, and policy-gradient examples covered here are all **model-free**.

09 Sampling & Trajectories

DEFINITION

Sampling — trying actions in order to generate experience. A **trajectory** is one resulting sequence of state, action, reward, state, action, reward...

With zero prior knowledge, the initial policy is uniform-random over actions. Most early trajectories go nowhere (e.g. a full -20 reward on a 20-step maze that never reaches the target), but by sheer luck a few succeed — and those are what learning bootstraps from.

10 The Monte Carlo Method

Given a successful trajectory, compute the return at every time step by working **backward** from the end: the last step's return is 0, and each earlier step's return is $\text{reward} + \gamma \times (\text{next step's already-computed return})$.

BACKWARD RETURN CALCULATION

$$G(t) = r(t) + \gamma \cdot G(t+1)$$

CORNER CASE

Even in a *successful* trajectory, most computed returns are still negative (most per-step rewards are -1) — only steps near the goal look good. Unsuccessful trajectories are even more negative, since they collect -1 for the full 20 steps and never reach the 0 reward.

DEFINITION

Monte Carlo method (per Wikipedia) — a computational algorithm relying on repeated random sampling to obtain numerical results. In RL: use full sampled trajectories to compute returns, then average them into value estimates.

11 Why Raw Returns Aren't Enough

Naive idea: push the probability of good-return actions up, bad-return actions down. The problem — with a softmax-style policy, the effect is **proportional**, not absolute.

CORNER CASE — worked comparison

Returns of $(-5, 0, 5)$ vs. $(5, 10, 15)$ vs. $(95, 100, 105)$ have the same absolute spread and ranking, but very different *proportional* effects: $10 \rightarrow 15$ is a 50% jump, while $100 \rightarrow 105$ is only 5%. The maze example is worse still — every return is negative, so there's no clean good/bad split to push on. This motivates subtracting a **baseline** (the expected return for that state), so what matters is being better/worse than average — leading directly to value functions.

12 Value Functions: V and Q

DEFINITION

State-value function, $V^\pi(s)$ — expected return if you're in state s and then follow policy π . **Action-value function**, $Q^\pi(s, a)$ — expected return if you're in state s , take action a , and then follow policy π .

RELATIONSHIP

$$V(s) = \sum(a) \pi(a|s) \cdot Q(s, a)$$

— V is just Q averaged over the policy's own action probabilities.

Optimal value functions $V^*(s)$, $Q^*(s, a)$ — the value under the best-possible action-selection at every step (“the best case scenario,” independent of any specific policy).

CORNER CASE

Unlike π and the world model — explicitly written as probability distributions — V and Q here are plain expected numbers, with no notion of variance/uncertainty. **Bayesian RL** and **distributional RL** do model that variance (think: risk-to-reward ratio), but the mainstream treatment covered here does not.

Three ways to improve the policy

1. **Pure policy gradient** (no baseline) — suffers the proportional-scaling problem above.
2. **Policy gradient + value-function baseline** — balances policy- and value-dependence; often the most powerful in practice (becomes actor-critic, §22).
3. **Greedy on Q** — ignore explicit action probabilities; always take $\arg\max_a Q(s, a)$. This is the purely value-based family (Monte Carlo $\rightarrow$ SARSA $\rightarrow$ Q-learning $\rightarrow$ DQN, next).

13 Generalized Policy Iteration & ϵ -greedy

CORNER CASE — cyclic dependency

Method 3 above creates a loop: Q is normally defined *relative to* a policy π , but if π is itself defined as “always pick $\arg\max Q$,” then Q depends on π *and* π depends on Q at the same time.

DEFINITION

Generalized Policy Iteration (GPI) — the repeating cycle of (1) improve the accuracy of the value function for the current policy, then (2) improve the policy using that updated value function, then repeat. Over many cycles $Q \rightarrow Q^*$ and $\pi \rightarrow$ the optimal policy.

Learning Q by incremental averaging

MONTÉ CARLO Q-UPDATE — moves the estimate a fraction α toward each new sample

$$Q(s, a) \leftarrow Q(s, a) + \alpha \cdot (G(t) - Q(s, a))$$

CORNER CASE

This is *not* a simple batch average — it's an incremental, recency-weighted nudge controlled by the learning rate α ($0.1 = 10\%$ in the worked demo). Every time the same (state, action) pair recurs, the estimate is nudged again toward the newest sample.

DEFINITION

Exploration — trying diverse/random actions to gather information. **Exploitation** — using current estimates to pick the best-known action. **ϵ -greedy** — with probability ϵ pick a uniformly random action, otherwise pick the greedy (argmax) action; ϵ is typically scheduled to *decay* over training (e.g. $0.9 \rightarrow 0.1$) as estimates become trustworthy.

CORNER CASE

ϵ -greedy is only needed for **greedy / value-based** methods — once you always take the argmax action, all randomness (and further exploration) disappears. **Policy-gradient** methods don't need this hack: keeping an explicit probability distribution means some randomness persists naturally.

14 Temporal Difference (TD)

► Watch this part — 36:23

Two problems with Monte Carlo

1. You must wait for the **entire episode to end** before any update — slow for long episodes, and it simply doesn't work for continuing (non-terminating) tasks.
2. The **credit assignment problem**: within one trajectory you can't tell which specific actions were the good decisions — you only have averaging over many samples to slowly tease that out.

DEFINITION

Credit assignment problem — determining the individual contribution of one action within a long sequence of many actions that led to an eventual outcome.

DEFINITION

Temporal difference — evaluate *action-by-action* instead of episode-by-episode: estimate a state-action pair's value relative to the (already-estimated) value of the very next state-action pair, using just **one real reward** as the correction signal, rather than waiting to sum all future rewards. A form of *bootstrapping*.

Terminal states are fixed at value 0 — an anchor that every other estimate is transitively pulled toward through the chain of one-step updates.

CORNER CASE — continuing tasks

With no terminal state there's no absolute anchor, so the *absolute magnitude* of value estimates can drift arbitrarily up or down during training. This is fine in practice, because policy improvement only needs the *relative* ordering of values (which action is better), not their absolute scale.

15 SARSA, Expected SARSA, Q-learning

The general TD update needs one open decision: which action to use for the *next* state when forming the target.

GENERAL TD UPDATE

$$Q(s(t), a(t)) \leftarrow Q(s(t), a(t)) + \alpha [r(t) + \gamma Q(s(t+1), a_{\text{next}}) - Q(s(t), a(t))]$$

Algorithm	a_next used	Policy type	Estimates
SARSA	the action the agent actually takes next	on-policy	Q_π
Expected SARSA	weighted average over all next actions, by $\pi(a' s')$	on-policy	Q_π
Q-learning	best possible next action, $\max Q(s', a')$	off-policy	Q^* directly

DEFINITION

On-policy — the behaviour policy (used to gather experience) and the target policy (being evaluated/improved) are the *same*. **Off-policy** — they differ: Q-learning acts using π but directly estimates the optimal Q^* , regardless of what action was actually taken.

CORNER CASE — sample efficiency

Ranking, least $\rightarrow$ most sample-efficient: **Monte Carlo** < **SARSA** < **Expected SARSA** < **Q-learning**. Q-learning benefits from both TD bootstrapping *and* off-policy optimal-target estimation. But off-policy learning isn't automatically superior in every situation — it has its own trade-offs beyond raw efficiency.

CORNER CASE — demo result

In a *small* maze, Monte Carlo vs. Q-learning is a modest gap. In a *larger* maze, Q-learning clearly reaches the target while Monte Carlo almost never does. Even after artificially handing Monte Carlo 50 successful warm-up episodes, it still failed to generalise afterward — a stark illustration of just how sample-inefficient Monte Carlo is on harder tasks.

16 The Bellman Optimality Equation

With enough samples, Q-learning's simple averaging update converges toward the true Bellman-optimal Q — even without ever having direct access to the environment's real transition probabilities (the world model); the samples themselves average out according to those hidden probabilities.

BELLMAN OPTIMALITY EQUATION FOR Q^*

$$Q^*(s, a) = E[r + \gamma \cdot \max_{a'} Q^*(s', a') \mid s, a]$$

DEFINITION

Bellman's Principle of Optimality (Richard Bellman, 1950s) — “An optimal policy has the property that whatever the initial state and initial decision are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision.” In plain terms: if a whole sequence of decisions is optimal, every sub-sequence of it, from any later point, must also be optimal on its own. (Analogy: full marks on a 3-part test requires full marks on each part.)

An analogous Bellman optimality equation exists for V^* too.

17 Deep Q-Networks (DQN)

► Watch this part — 50:55

Tables can only hold finitely many discrete (state, action) entries — impossible for continuous state spaces, needed for nearly any real physical task (angles, velocities, positions). **DQN = Q-learning with the table replaced by a neural network function approximator.** The network takes a state and outputs a Q-value for every possible discrete action at once (more efficient than one call per action). Trained via the Q-learning TD target, using backprop + gradient descent instead of table averaging.

CORNER CASE — discrete actions only

DQN still only supports a **discrete, enumerable** set of actions, because it's value-based: you must be able to take an argmax over “all possible actions,” which is intractable for a continuous/infinite action space. This is exactly what motivates policy-gradient methods (§18) for continuous control.

Worked example — Lunar Lander

- **State** (8 continuous variables): x, y position · x, y velocity · angle · angular velocity · two boolean leg-contact flags.
- **Action** (4 discrete choices): no engine, OR exactly one of {left, right, down} engine (only one fires at a time).
- **Reward shaping**: higher for being close to the pad / moving slowly / staying horizontal; small penalty per engine firing (fuel efficiency); bonus for leg-ground contact; large penalty for crashing; large bonus for a safe landing.
- **Hyperparameters**: $\gamma = 0.999$ (near 1, since episodes have many small time steps), learning rate $\alpha = 0.001$, plus ϵ -greedy exploration. Network: hidden layers of 64 nodes each.

CORNER CASE — imperfect interpretability

Probing the trained network with hand-set states reveals mostly sensible behaviour (fires the opposite-side engine to re-centre, fires the down engine when too low) — but not every input combination produces symmetric/intuitive behaviour. A working policy can still have non-intuitive blind spots.

DQN variants & flagged-but-unexplained tricks

- Double DQN, Dueling DQN, Rainbow DQN, etc. — all fundamentally “Q-learning + neural network,” differing in implementation tricks.

CORNER CASE — terms to know, not derived here

Even the original 2013 DQN paper (“Playing Atari with Deep Reinforcement Learning”) relies on a **target network** and a **replay buffer** — flagged as important but out of scope for a deep dive. Know that these terms exist.

18 Policy Gradients

► Watch this part — 58:46

Instead of picking greedily from a value function, directly maintain and adjust a probability distribution over actions — the policy itself. This naturally extends to a **continuous** probability distribution (e.g. a Gaussian defined by a mean and standard deviation), essential for real, physical continuous-control tasks.

CORNER CASE — no ground-truth target

Value functions (V , Q) always have a concrete numeric target to regress toward — a clean loss-minimisation setup. The **policy** function has no such target: you're not fitting a known correct answer, you're reinforcing what seems to work. So instead of *minimising a loss*, policy-gradient methods *maximise an objective function*.

19 The Objective Function $J(\theta)$

DEFINITION

Objective function, $J(\theta)$ — a score for the *whole* policy's parameters θ (not a specific state or action), representing overall performance. Equivalent-in-spirit definitions: expected total reward across all trajectories · V of the initial state · average V across all states · average reward per time step.

CORNER CASE

Unlike Q or V , actually *computing* J 's numeric value isn't very useful on its own (it's basically the total-reward learning curve). The real value of J is that **differentiating** it with respect to θ , via the Policy Gradient Theorem, yields a usable training signal.

POLICY GRADIENT THEOREM — general form

$$\nabla(\theta) J(\theta) = E[\sum(t) \nabla(\theta) \log \pi(\theta)(a(t)|s(t)) \cdot (\text{how good was } a(t))]$$

- $\nabla J(\theta)$ — how to nudge parameters θ to increase performance J .
- $\nabla \log \pi(a|s)$ — how to nudge θ to increase the probability of the action actually taken.
- The weighting term — a scalar for “how good was this action” (several definitions, next section).
- The outer expectation/sum — average this nudge across all sampled time steps.

Plain-language reading: increase the probability of actions that did well, decrease the probability of actions that did poorly, weighted by how good/bad each one was — the complicated notation is just a formal way of saying exactly that.

20 “How Good Was This Action?” — Advantage & Variance

Progression of choices for the weighting term, each fixing a weakness of the one before:

1. **$G(t)$** (raw Monte Carlo return) — simplest, but high variance (different states have different typical averages, so the scale is inconsistent — the exact problem from §11).
2. **$G(t) - V(s)$** — subtract the state-value baseline, so it's “better/worse than average for this state.” Still needs a full episode.
3. **$Q(s,a)$ or $Q(s,a) - V(s)$** — swap in the action-value function to enable TD (no full-episode wait) — but naively needs training two separate networks (Q and V).
4. **$r + \gamma V(s')$** — algebraically equal to $Q(s,a)$ by definition, so only a single V -network is needed. Note: $r + \gamma V(s') - V(s)$ is exactly the **TD error δ** (re-appears under Neuroscience, §24).
5. **Generalized Advantage Estimation (GAE)** — the most advanced/powerful variant (derivation out of scope here).

DEFINITION

Advantage — how much better (or worse) a specific action is compared to the average/expected outcome in that state, rather than evaluating it in isolation. This is what reduces variance for the policy-gradient update.

CORNER CASE

Variance wasn't a major concern for the earlier value-based methods (Q-learning/DQN), since those simply take the argmax action regardless of scale. It matters here specifically because policy-gradient updates adjust probabilities *proportionally*, so a noisy signal directly corrupts the whole distribution.

21 Discrete vs. Continuous Policy-Gradient Mechanics

Action space	How the update is applied
Discrete	Push the chosen action's probability up or down directly; with a softmax output, the other probabilities automatically renormalise to sum to 1.
Continuous (Gaussian)	Drag the distribution's mean toward the taken action, proportional to distance from the mean. Adjust the standard deviation: increase it if the action was far from the mean, decrease it if close. Both nudges are scaled by the action's advantage.

22 Actor-Critic, TRPO & PPO

DEFINITION

Actor-critic — any policy-gradient method whose “how good was this action” evaluation is computed fully via temporal difference (bootstrapping) rather than waiting for a full Monte-Carlo return. The **actor** is the policy function (picks/acts); the **critic** is the value function(s) (evaluates, to help the actor improve).

TRPO (Trust Region Policy Optimization) and **PPO** (Proximal Policy Optimization) don't directly optimise the raw gradient form above — they use **surrogate objective functions**: differently-formulated objectives that, empirically, train more reliably when optimised by a neural network, while still following the same overall actor + critic structure.

CORNER CASE — scope note

The TRPO/PPO math itself is explicitly *not derived* in the source — know the names and their role (more stable surrogate objectives), not the equations.

Demos referenced

- **Lunar Lander via PPO** — two networks: one outputs action probabilities (actor), one outputs a single state-value $V(s)$ (critic). Retains built-in randomness in action choice — unlike DQN's artificial ϵ -greedy.
- **BipedalWalker via Soft Actor-Critic (SAC)** — a continuous-action task: one continuous probability distribution per joint. Here the advantage is computed via an *averaged action-value* Q rather than V — a reminder that the “which advantage flavour” choice (§20) genuinely varies across real implementations.

23 RL & Neuroscience

► Watch this part — 1:12:08

Two well-established correlations between RL and how the brain works: (1) temporal difference $\leftrightarrow$ dopamine, and (2) actor-critic $\leftrightarrow$ dorsal-lateral vs. ventral striatum.

24 Dopamine & the TD Error

DEFINITION

Reward Prediction Error hypothesis of dopamine — dopamine neuron activity encodes the *error* between expected and actual reward (a prediction-error signal), not reward itself.

Key experiment — Wolfram Schultz et al., 1990s (monkeys)

A monkey presses a lever after a light cue and receives apple juice.

- Naively, dopamine spikes when the juice arrives — looks like “dopamine = reward.”
- But once the monkey *learns* the light predicts juice, the spike shifts **earlier** — to the predictive light — and stops firing for the now-expected juice itself.
- Two-lever version (instruction cue $\rightarrow$ trigger cue $\rightarrow$ juice): as learning progresses, the spike shifts even further back, from the trigger cue all the way to the earliest reliable predictive cue.

CORNER CASE — negative prediction error

When the monkey pressed the wrong lever (expected juice, got none), dopamine neurons showed a sharp **drop** in activity — a negative prediction error, consistent with dopamine encoding “better/worse than expected,” not simply “reward present.”

DEFINITION

TD error, $\delta(t)$ — the difference between what was originally expected and what is now expected, given a newly-received reward.

TD ERROR — for the V function

$$\delta(t) = r(t) + \gamma \cdot V(s(t+1)) - V(s(t))$$

This δ is the *same quantity* used to drive TD value-function updates (§14) — the mathematical “signal that guides learning” is identical to what neuroscientists observe as the dopamine signal. The backward shift in timing occurs because TD propagates reward information back through the chain of states one step at a time; once a consistent predictive cue exists earlier in the sequence, the “surprise” migrates back to that earliest

reliably-predictive point — there being nothing earlier for it to propagate to before such a cue exists.

25 Actor-Critic in the Brain

The **striatum** is broadly involved in motor/action planning, decision-making, motivation, reinforcement, and reward perception.

- **Dorsal-lateral striatum** $\approx$ the actor (action selection & execution).
- **Ventral striatum** $\approx$ the critic (evaluation & prediction, to guide the actor).

Supporting evidence (a 2008 review cited in the source): just as in an actor-critic RL algorithm the TD error feeds into *both* the actor and the critic, dopamine signals biologically project to *both* the dorsal-lateral and ventral striatum.

CORNER CASE / caveat

The broader supporting neuroscience evidence beyond this specific point is far more technical/detailed and is explicitly not covered in depth in the source.

26 Limitations of Current RL

► Watch this part — 1:20:26

Reference: the widely-cited 2018 article “*Deep Reinforcement Learning Doesn't Work Yet*” — many of its points are still considered relevant today.

CORNER CASE — unreliability

Identical code and hyperparameters, run multiple times with only the random seed changed, can produce wildly different outcomes — e.g. succeeding roughly half the time and failing the rest. The creator reports this personally: repeated runs of Lunar Lander trained with a policy-gradient method varied enormously, which is why a pre-trained model was downloaded from the internet for the demo instead of training fresh, for time reasons.

CORNER CASE — sample inefficiency

Rainbow DQN needed roughly **18 million frames** to reach human-level performance on Atari games. At 60 frames/second that's about **83 hours** — for a game a human might learn in minutes. Some research papers report needing 10-100+ million frames for other tasks. Fine for cheaply-simulated video games; impractical for physical-world settings where that much experience can't be cheaply generated.

27 Future Direction 1 — Model-Based RL

Recap: a world model predicts the next state and reward given the current state and action — it encodes “how the world works,” letting an agent imagine/simulate outcomes instead of learning purely from blind trial and error.

CORNER CASE — why model-free is so sample-hungry

Analogy: a human with zero physical intuition (no concept of gravity, contact, inertia) trying to learn “don't let this tennis ball touch the table” would need roughly 300 blind trials just to learn one half of the table, then another 300 for the other half. A human with physical intuition needs far fewer. The complete lack of a world model is exactly why model-free methods (everything covered up to §22) require 10M-100M+ training samples.

Obtaining a world model

- **Hand-crafted / given** — e.g. known physics equations, or known board-game rules.
- **Learned** — by the agent, simultaneously alongside its policy/value function.

Using a world model

- **Train on imagined experience** generated by the model — analogous to the well-known finding that mental rehearsal (e.g. imagining shooting free throws) can

improve real performance.

- **Plan ahead** during actual execution, by simulating different possible action sequences and searching for the best one — e.g. **Monte Carlo Tree Search (MCTS)**: simulate how different future branches might play out, and search for the most promising path.

DEFINITION

Monte Carlo Tree Search (MCTS) — a planning method that simulates how different possible future action sequences might branch out, searching for the best option.

Real progression: DeepMind's Go/Atari family

AlphaGo → AlphaGo Zero → AlphaZero → MuZero, expanding capability from Go, to other board games (chess, shogi), to Atari.

CORNER CASE — key distinction

AlphaGo, AlphaGo Zero, and AlphaZero are all **given** a model (known game rules) and use it only for **planning**. Only **MuZero learns its own** world model from scratch, and uses that learned model for *both* imagined-experience training *and* planning.

Dreamer series (Dreamer, Dreamer V2, Dreamer V3) — each learns its own world model and uses it *only* for training on imagined experience (not for explicit planning/search). Dreamer V3 notably obtained diamonds in a modified Minecraft — a benchmark known for being a hard, sparse-reward, long-horizon task.

28 Future Direction 2 — Imitation & Inverse RL

Many everyday skills (tying shoelaces, folding a shirt, shooting a basketball) are normally learned by *watching* a demonstrator — not by blind trial-and-error (you wouldn't expect a 3-year-old to learn shoelace-tying purely through trial and error).

DEFINITION

Imitation learning — learning a policy (state $\rightarrow$ action mapping) directly from an expert's example trajectories, *without* using any reward signal at all.

CORNER CASE — reward hacking, and why imitation avoids it

Designing a good reward function is often surprisingly hard. Example: reward a driving agent simply for “crossing the finish line,” and it may learn to repeatedly cross back and forth to keep collecting reward, instead of actually completing a lap. Full safe-driving behaviour (lane changes, signalling) is even harder to hand-specify as a reward. Imitation learning sidesteps reward design entirely by learning directly from demonstration.

DEFINITION

Behavioural cloning — the simplest form of imitation learning: fit a policy (table or neural network) directly to the expert's recorded (state, action) pairs via supervised learning.

CORNER CASE — distribution shift

Behavioural cloning works fine as long as the agent stays exactly *on* the demonstrated trajectory — but the moment it drifts even slightly off-course, it has no training data for that situation and tends to drift *further* away (compounding errors).

DEFINITION

Dagger (Dataset Aggregation) — an improvement on behavioural cloning: the agent deliberately adds noise to generate realistic “slightly off-course” situations, a human labels the correct corrective action for each, and this gets folded back into training. More robust, at the cost of ongoing tedious human labelling.

Real demos mentioned: a robot learning to catch a ball in a cup via imitation (improves over ~100 trials); a robot dog learning to walk by imitating a real dog; a surgical robot learning to suture from human demonstration footage.

DEFINITION

Inverse Reinforcement Learning (Inverse RL) — rather than copying the demonstrated behaviour directly, infer the underlying *reward function* that would explain/produce the expert's observed behaviour — the implicit goal, not just the surface actions.

CORNER CASE — imitation learning vs. inverse RL

Imitation learning mimics the *behaviour*. Inverse RL tries to recover the *intent* (reward function) behind that behaviour — which, once recovered, could in principle be used to train a fresh RL agent from scratch via ordinary RL.

Example: “Inverse Q-learning” learns the Q-function (and subsequently the reward function) from expert demonstrations — used to learn Atari gameplay, Minecraft movement, and simulated robot-arm control from as few as 20-30 expert demos.

29 Recap

► Watch this part — 1:31:58

1. Framed a task as a **Markov Decision Process**, applied to a grid-maze example.
2. Solved it with **Monte Carlo**, then improved with **Temporal Difference** — specifically **Q-learning**.
3. Added a neural network to Q-learning → **Deep Q-Networks**, handling continuous *state* spaces (still discrete actions).
4. **Policy Gradients** extended this further to continuous *action* spaces too.
5. Explored two RL ↔ Neuroscience correlations: dopamine/TD error, and actor-critic/striatum.
6. Closed on current shortcomings (unreliability, sample inefficiency) and two active research directions: **model-based RL** and **imitation/inverse RL**.

30 Glossary — Quick Reference

Agent

Whatever you can directly control.

Environment

Whatever you cannot directly control, but interact with via the agent.

Action

The agent's influence on the environment.

State

The environment's influence on the agent (its observation).

Reward

Scalar feedback signal after each action; more positive = better.

MDP

Discrete repeating sequence of state, action, reward — the standard RL problem formalism.

Episode

A trajectory that terminates.

Markov property

Each state depends only on the immediately preceding state/action.

Policy $\pi(a|s)$

Probability of taking action a in state s ; stochastic by design.

Return $G(t)$

Discounted cumulative future reward from time t .

Discount factor γ

Weighting (0-1) that reduces the value of future rewards.

World model

Function giving next state/reward probabilities given state and action.

Model-based / model-free

Whether the agent has (or learns) access to a world model.

Trajectory

One sampled sequence of state-action-reward experience.

Sampling

Trying actions to generate experience.

Value function $V_\pi(s)$

Expected return from state s under policy π .

Action-value function $Q_\pi(s,a)$

Expected return from taking a in s , then following π .

Optimal value functions V^*, Q^*

Value under the best possible action-selection ("best case").

Baseline

A reference value (e.g. $V(s)$) subtracted from a return to reduce variance.

Generalized Policy Iteration (GPI)

Cycle of evaluating the value function, then improving the policy, repeated.

Exploration / exploitation

Trying new actions to learn vs. using known-good actions to perform well.

 ϵ -greedy

Random action with probability ϵ , greedy action otherwise; ϵ typically decays.

Monte Carlo method

Learning from full sampled trajectories/episode returns.

Credit assignment problem

Determining which individual action(s) caused an eventual outcome.

Temporal difference (TD)

Bootstrapped, one-step learning using the next state's current estimate.

SARSA

On-policy TD control using the actually-taken next action.

Expected SARSA

On-policy TD control using the policy-weighted average over next actions.

Q-learning

Off-policy TD control using the best possible next action (max).

On-policy / off-policy

Whether the behaviour policy and target policy are the same.

Sample efficiency

How few episodes/steps are needed to learn a task well.

Bellman optimality equation

Recursive equation defining the optimal value function.

Function approximator

A model (e.g. neural network) standing in for a lookup table.

DQN

Q-learning with a neural network approximating Q; discrete actions only.

Target network / replay buffer

Standard DQN stabilisation tricks (named, not derived in the source).

Objective function $J(\theta)$

Overall performance score for the whole policy, maximised via gradient ascent.

Policy gradient theorem

Formula for $\nabla J(\theta)$ that drives policy-gradient training.

Advantage

How much better/worse an action is than the average for that state.

Actor-critic

Policy-gradient method whose evaluation is fully TD-based; actor = policy, critic = value function.

TRPO / PPO

Policy-gradient algorithms using more stable “surrogate” objective functions.

Generalized Advantage Estimation (GAE)

Advanced, low-variance advantage estimator.

TD error δ

Difference between the old and new (reward-updated) value estimate.

Reward prediction error hypothesis

Dopamine encodes prediction error, not reward itself.

Striatum (dorsal-lateral / ventral)

Brain regions functionally analogous to the actor / critic.

Monte Carlo Tree Search (MCTS)

Planning by simulating and searching over possible future branches.

Behavioural cloning

Supervised-learning fit of a policy to expert (state, action) pairs.

Dagger

Behavioural cloning improved with human-labelled corrections on off-course states.

Imitation learning

Learning a policy from expert demonstrations, without a reward signal.

Inverse RL

Inferring the reward function that explains an expert's demonstrated behaviour.

31 Formula Sheet

ALL KEY EQUATIONS IN ONE PLACE

Return: $G(t) = \sum_{k \geq 0} \gamma^k \cdot r(t+k)$

Backward return: $G(t) = r(t) + \gamma \cdot G(t+1)$

V-Q relationship: $V(s) = \sum(a) \pi(a|s) \cdot Q(s,a)$

MC Q-update: $Q(s,a) \leftarrow Q(s,a) + \alpha \cdot (G(t) - Q(s,a))$

General TD update: $Q(s(t),a(t)) \leftarrow Q(s(t),a(t)) + \alpha \cdot [r(t) + \gamma \cdot Q(s(t+1),a_{\text{next}}) - Q(s(t),a(t))]$

Bellman optimality (Q^*): $Q^*(s,a) = E[r + \gamma \cdot \max_{a'} Q^*(s',a')]$

TD error: $\delta(t) = r(t) + \gamma \cdot V(s(t+1)) - V(s(t))$

Policy gradient theorem: $\nabla J(\theta) = E[\sum(t) \nabla \log \pi(a(t)|s(t)) \cdot \text{advantage}]$

Advantage (TD form): $A(s,a) \approx r + \gamma \cdot V(s') - V(s) (= \delta(t))$

32 Corner-Case Checklist — Pre-Exam Scan

- Agent/environment boundary and action time-scale are **arbitrary** — choose the framing, don't assume one.
- Two reward-indexing conventions exist across sources ($r(t)$ vs. $r(t+1)$) — check which one a formula uses.
- $\gamma = 1$ is only valid for **terminating** episodes; otherwise the return sum diverges.
- The Markov property depends on *what you put in the state*, not on the real world itself (velocity example).
- Raw returns as a policy-gradient signal fail because the effect is proportional, not absolute — hence baselines.
- V and Q here are point estimates with no variance modelling (unlike Bayesian/distributional RL).
- Greedy-on- Q creates a policy↔value cyclic dependency, resolved via GPI.
- ϵ -greedy is a patch needed only for greedy/value-based methods, not for policy gradients.
- Monte Carlo cannot be used on continuing (non-terminating) tasks.
- Terminal states anchor TD values at 0; continuing tasks have no such anchor, so absolute values can drift (relative ordering still works).
- On-policy (SARSA, Expected SARSA) evaluates the behaviour policy; off-policy (Q-learning) evaluates the optimal policy directly.
- Sample efficiency order: MC < SARSA < Expected SARSA < Q-learning.
- DQN is value-based → discrete actions only; exactly why continuous control needs policy gradients.
- Even standard DQN quietly relies on a target network and replay buffer — know the names.
- A trained network's behaviour can be inconsistent/asymmetric even when it “works” overall.
- $J(\theta)$'s exact definition doesn't matter much; what matters is its gradient.
- The “advantage” term has at least five common forms ($G(t)$, $G(t)-V$, Q , $Q-V$, $r+\gamma V'-V$, GAE) — know the progression and why each step was introduced.
- TRPO/PPO use surrogate objectives, not the raw policy-gradient formula — names to know, math not required here.
- Reward-hacking example (looping across a finish line) shows why imitation learning can be preferable to designing a reward function.
- Behavioural cloning fails under distribution shift; DAGger fixes this but needs ongoing human labelling.
- Imitation learning copies behaviour; inverse RL recovers the reward function behind the behaviour — different goals.

- Only MuZero (of the AlphaGo family) learns its own world model; AlphaGo/Zero/Zero-era models are all given the rules.
- Dreamer-series models learn a world model but use it only for imagined training, not explicit planning/search.
- RL training runs are seed-sensitive/unreliable, and sample-inefficient (Rainbow DQN: $\sim 18\text{M}$ frames ≈ 83 hours for Atari human-level play).

Notes compiled from a single long-form reinforcement-learning lecture (transcript-sourced). Section headings mirror the video's own chapter structure; timestamp links jump to the source video's chapter markers.